Topic 5: DataFrames

Introduction to Pandas DataFrames

Learning Outcomes:

- Understand why pandas DataFrames improve data handling
- Import pandas using the standard convention
- Construct DataFrames from lists and CSV files
- Use basic DataFrame metadata methods
- Apply 4 principles for working with DataFrames

Opening Problem

In Topic 4, you learned to work with CSV files using the `csv` module.

You can read student records from a CSV file:

```
import csv

f = open("students.csv", "r", newline="")
reader = csv.reader(f)

for row in reader:
    print(row)

f.close()
```

Output:

```
['Name', 'Score', 'Grade']
['Ali', '85', 'A']
['Sara', '92', 'A+']
['Ahmad', '78', 'B']
```

But what if you need to analyze this data?

The Problem With csv.reader()

Task: Find the average score

```
import csv

f = open("students.csv", "r", newline="")
reader = csv.reader(f)

scores = []
for row in reader:
    if row[0] != "Name":  # Skip header
        scores.append(int(row[1]))  # Convert string to int

f.close()

average = sum(scores) / len(scores)
print(f"Average: {average}")
```

Problems:

- Manual header skipping
- Manual string-to-int conversion
- Need to know which column contains scores (index 1)
- Separate code needed for every analysis task

There must be a better way.

The Solution: DataFrames

All related data in one unified structure:

```
import pandas as pd

students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad", "Fatimah"],
    'Score': [85, 92, 78, 88],
    'Grade': ["A", "A+", "B", "A"]
})

print(students)
```

Output:

```
        Name  Score Grade
0        Ali     85     A
1       Sara     92    A+
2      Ahmad     78     B
3    Fatimah     88     A
```

Data stays synchronized. One structure to manage.

Why DataFrames?

Problem with csv.reader()	Solution with DataFrames
Manual iteration for every task	Built-in analysis methods
String data types	Automatic type detection
Index-based column access	Name-based column access
Manual header handling	Automatic header recognition
Complex code for simple tasks	Simple, readable operations

DataFrames automate what you did manually in Topic 4.

Where Pandas Fits

Pandas: Python's data analysis library

```
┌──────────────────────────────────────────────┐
│              Python Standard Library         │
│   (lists, dicts, built-in functions)         │
└──────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────┐
│                   Pandas                     │
│   (DataFrames, Series, data analysis)        │
└──────────────────────────────────────────────┘
```

Pandas provides:

- High-level data structures (DataFrames, Series)
- Tools for reading and writing data

Two Core Pandas Structures

Structure	Dimensions	Description	Visual
Series	1D	Single column or row of labeled data	One column from a table
DataFrame	2D	Table with rows and columns	Complete table

This topic focuses on DataFrames — the most commonly used structure.

Series vs DataFrame

Series (1D):

```
scores = pd.Series([85, 92, 78, 88])
print(scores)
```

```
0    85
1    92
2    78
3    88
dtype: int64
```

DataFrame (2D):

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara"],
    'Score': [85, 92]
})
```

```
   Name  Score
0   Ali     85
1  Sara     92
```

Visual: Series vs DataFrame

```
Series (1D):                DataFrame (2D):

Index  Values               Name     Score  Grade
  0      85                 Ali        85     A
  1      92                 Sara       92     A+
  2      78                 Ahmad      78     B
  3      88
```

Think of:

- Series = One column from a spreadsheet
- DataFrame = Entire spreadsheet with multiple columns

Structure of a DataFrame

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78]
})
```

Output:

```
     Name  Score
0     Ali     85
1    Sara     92
2   Ahmad     78
```

Let's break down each component...

DataFrame Components

```
        ← Columns →
      Name     Score
    ┌──────────────────┐
 0  │ Ali       85     │  ← Row (Record)
 1  │ Sara      92     │  ← Row (Record)
 2  │ Ahmad     78     │  ← Row (Record)
    └──────────────────┘
   ↑
 Index
```

Component	Description	Example
Columns	Vertical fields (attributes)	`Name`, `Score`
Rows	Horizontal records (observations)	`["Ali", 85]`
Index	Row labels (0, 1, 2...)	`0`, `1`, `2`

Exercise 1: Identify DataFrame Parts

Given this DataFrame:

```
      Product  Price  Stock
0      Laptop   2500     15
1       Mouse     45    120
2    Keyboard    120     58
```

Identify:

1. How many columns?
2. How many rows?
3. What are the column names?
4. What is the index range?
5. What is the value at row 1, column Price?

Deep Dive: Working with DataFrames

Core operations:

1. Importing pandas
2. Constructing DataFrames from lists
3. Constructing DataFrames from CSV
4. Accessing DataFrame metadata

Let's explore each...

5.1: Importing Pandas

Standard import convention:

```
import pandas as pd
```

Breaking it down:

Part	Meaning
`import pandas`	Load the pandas library
`as pd`	Create alias `pd` (shorter to type)

Why `pd`?

- Universal convention in the data science community
- Makes code consistent across projects

5.2: Constructing from Lists

Two common methods:

Method 1: Dictionary of lists (recommended)

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78]
})
```

Method 2: List of lists with column names

```
data = [
    ["Ali", 85],
    ["Sara", 92],
    ["Ahmad", 78]
]
students = pd.DataFrame(data, columns=['Name', 'Score'])
```

Method 1: Dictionary of Lists

Syntax:

```
df = pd.DataFrame({
    'column1': [value1, value2, ...],
    'column2': [value1, value2, ...]
})
```

Example:

```
products = pd.DataFrame({
    'Product': ["Laptop", "Mouse", "Keyboard"],
    'Price': [2500, 45, 120],
    'Stock': [15, 120, 58]
})
print(products)
```

Output:

```
    Product  Price  Stock
0    Laptop   2500     15
1     Mouse     45    120
2  Keyboard    120     58
```

Method 2: List of Lists

Syntax:

```
data = [[row1_val1, row1_val2], [row2_val1, row2_val2]]
df = pd.DataFrame(data, columns=['col1', 'col2'])
```

Example:

```
data = [
    ["Laptop", 2500, 15],
    ["Mouse", 45, 120],
    ["Keyboard", 120, 58]
]
products = pd.DataFrame(data, columns=['Product', 'Price', 'Stock'])
print(products)
```

Output:

```
    Product  Price  Stock
0    Laptop   2500     15
1     Mouse     45    120
2  Keyboard    120     58
```

Exercise 2: Create a DataFrame

Create a DataFrame for this data using the dictionary method:

City	Temperature	Humidity
Kuala Lumpur	32.5	80
Tokyo	28.0	65
Dubai	38.5	45

Write the code yourself before continuing...

5.3: Constructing from CSV

Building on Topic 4: From `csv.reader()` to `pd.read_csv()`

Topic 4 approach (manual):

```
import csv

f = open("students.csv", "r", newline="")
reader = csv.reader(f)
content = []
for row in reader:
    content.extend([row])
f.close()
```

Topic 5 approach (automated):

```
import pandas as pd

students = pd.read_csv("students.csv")
```

Pandas does in 1 line what took 6 lines manually.

`pd.read_csv()` — The Better Way

Read CSV file into DataFrame:

```
students = pd.read_csv("students.csv")
```

What pandas handles automatically:

- Opens the file
- Reads all rows
- Parses CSV structure
- Creates column headers from first row
- Closes the file
- Returns organized DataFrame

File path options:

Location	Example
Same folder	`pd.read_csv("data.csv")`
Relative path	`pd.read_csv("data/students.csv")`
Absolute path	`pd.read_csv("c:/users/name/data.csv")`

Note: Use forward slashes `/` even on Windows.

Example: Reading CSV

Given `products.csv`:

```
Product,Price,Stock
Laptop,2500,15
Mouse,45,120
Keyboard,120,58
```

Code:

```python
import pandas as pd

products = pd.read_csv("products.csv")
print(products)
```

Output:

```
    Product  Price  Stock
0    Laptop   2500     15
1     Mouse     45    120
2  Keyboard    120     58
```

First row automatically becomes column names.

5.4: DataFrame Metadata

Metadata = Data about data

Three categories:

1. **Attributes** — Properties (no parentheses): `df.shape`, `df.columns`, `df.index`
2. **Methods** — Actions (with parentheses): `df.info()`, `df.head()`, `df.tail()`

Important distinction:

- Attributes: `df.shape` (NO parentheses)
- Methods: `df.head()` (WITH parentheses)

Metadata: `shape` (Attribute)

What it is: Returns dimensions as (rows, columns)

Syntax:

```
df.shape
```

Important: NO parentheses! It's an attribute, not a method.

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78]
})

print(students.shape)
```

Output:

```
(3, 2)
```

Reading: 3 rows, 2 columns

`shape` — Format Explained

```
print(students.shape)  # (3, 2)
```

Breaking it down:

```
(rows, columns)
  |       |
  3       2
```

Remember: Rows first, columns second (like matrix notation)

Common use:

```
rows, cols = students.shape
print(f"Dataset has {rows} rows and {cols} columns")
```

Metadata: `columns` (Attribute)

What it is: Returns column names

Syntax:

```
df.columns
```

Important: NO parentheses!

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara"],
    'Score': [85, 92],
    'Grade': ["A", "A+"]
})

print(students.columns)
```

Output:

```
Index(['Name', 'Score', 'Grade'], dtype='object')
```

`columns` — Converting to List

Get column names as a regular list:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara"],
    'Score': [85, 92]
})

col_list = list(students.columns)
print(col_list)
```

Output:

```
['Name', 'Score']
```

Use case: Iterate over column names, check if column exists

Metadata: `index` (Attribute)

What it is: Returns row labels

Syntax:

```
df.index
```

Important: NO parentheses!

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78]
})

print(students.index)
```

Output:

```
RangeIndex(start=0, stop=3, step=1)
```

Default index: Starts at 0, increments by 1

Metadata: `info()` (Method)

What it does: Displays DataFrame structure, data types, memory usage

Syntax:

```
df.info()
```

Important: WITH parentheses! It's a method.

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78],
    'Grade': ["A", "A+", "B"]
})

students.info()
```

`info()` — Output Explained

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Score   3 non-null      int64
 2   Grade   3 non-null      object
dtypes: int64(1), object(2)
memory usage: 200.0+ bytes
```

Let's break down each line...

`info()` — Line 1: DataFrame Class

```
<class 'pandas.core.frame.DataFrame'>
```

Meaning: Confirms this is a pandas DataFrame object

Why it matters: Verifies you're working with the correct data structure

`info()` — Line 2: Index Range

```
RangeIndex: 3 entries, 0 to 2
```

Breaking it down:

Part	Meaning
`RangeIndex`	Default numeric index (0, 1, 2...)
`3 entries`	Total number of rows
`0 to 2`	Index starts at 0, ends at 2

Use case: Quickly check how many rows of data you have

`info()` — Line 3: Column Summary

```
Data columns (total 3 columns):
```

Meaning: This DataFrame has 3 columns total

Use case: Verify expected number of columns loaded correctly

`info()` — Lines 4-7: Column Details

```
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Score   3 non-null      int64
 2   Grade   3 non-null      object
```

Each column shows:

Part	Meaning	Example
#	Column index position	`0`, `1`, `2`
Column	Column name	`Name`, `Score`, `Grade`
Non-Null Count	How many rows have values (not missing)	`3 non-null` = all 3 rows have data
Dtype	Data type pandas assigned	`object`, `int64`

`info()` — Understanding Dtype

Dtype = Data Type

Common pandas data types:

Dtype	What It Stores	Example Values
`object`	Text/strings (or mixed types)	`"Ali"`, `"A+"`
`int64`	Integers (whole numbers)	`85`, `92`, `78`
`float64`	Decimals	`3.14`, `88.5`
`bool`	True/False values	`True`, `False`
`datetime64`	Dates and times	`2024-01-15`

In our example:

- `Name` → `object` (strings)
- `Score` → `int64` (integers)
- `Grade` → `object` (strings)

`info()` — Line 8: Dtype Summary

```
dtypes: int64(1), object(2)
```

Breaking it down:

Part	Meaning
`int64(1)`	1 column has integer type
`object(2)`	2 columns have object type

Use case: Quick overview of data type distribution

Why it matters: Helps identify if types need conversion (e.g., strings that should be numbers)

`info()` — Line 9: Memory Usage

```
memory usage: 200.0+ bytes
```

Meaning: This DataFrame consumes approximately 200 bytes in memory

Why it matters:

- Large datasets may consume gigabytes
- Helps you monitor memory when working with big data
- The `+` indicates this is an estimate

Use case: Check if dataset fits in available RAM

`info()` — Complete Reading Example

Given this output:

```
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      1000 non-null   object
 1   Age       998 non-null    int64
 2   Salary    995 non-null    float64
 3   Active    1000 non-null   bool
 4   HireDate  1000 non-null   datetime64
dtypes: bool(1), datetime64(1), float64(1), int64(1), object(1)
memory usage: 39.2+ KB
```

Reading:

- 1000 rows total
- 5 columns
- `Age` has 2 missing values (998 out of 1000)
- `Salary` has 5 missing values (995 out of 1000)
- Mix of data types (text, numbers, dates, booleans)
- Uses about 39 KB of memory

Metadata: `head()` (Method)

What it does: Returns first N rows (default: 5)

Syntax:

```
df.head()       # First 5 rows
df.head(n)      # First n rows
```

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad", "Fatimah", "Hassan", "Nurul"],
    'Score': [85, 92, 78, 88, 95, 82]
})

print(students.head(3))
```

Output:

```
     Name  Score
0     Ali     85
1    Sara     92
2   Ahmad     78
```

Metadata: `tail()` (Method)

What it does: Returns last N rows (default: 5)

Syntax:

```
df.tail()       # Last 5 rows
df.tail(n)      # Last n rows
```

Example:

```
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad", "Fatimah", "Hassan", "Nurul"],
    'Score': [85, 92, 78, 88, 95, 82]
})

print(students.tail(3))
```

Output:

```
       Name  Score
3   Fatimah     88
4    Hassan     95
5     Nurul     82
```

Metadata Methods Summary

Method/Attribute	Type	Parentheses?	Returns	Use Case
`shape`	Attribute	NO	(rows, cols)	Check dimensions
`columns`	Attribute	NO	Column names	List all fields
`index`	Attribute	NO	Row labels	Check row range
`info()`	Method	YES	Structure summary	Explore new data
`head(n)`	Method	YES	First n rows	Preview data
`tail(n)`	Method	YES	Last n rows	Check last entries

Critical difference: Attributes have NO parentheses, methods have parentheses.

Exercise 3: Explore a DataFrame

Given this code:

```
products = pd.DataFrame({
    'Product': ["Laptop", "Mouse", "Keyboard", "Monitor", "Webcam"],
    'Price': [2500, 45, 120, 800, 150],
    'Stock': [15, 120, 58, 32, 78]
})
```

Write code to:

1. Print the dimensions
2. Print the column names
3. Display the first 2 rows
4. Display the last 2 rows
5. Show the DataFrame structure using `info()`

4 Principles for Working with DataFrames

Beyond syntax — professional practices:

1. **Standard Import Alias** — Always use `pd`
2. **Dictionary Construction** — Explicit column names
3. **Verify Shape** — Check dimensions after loading
4. **Consistent Naming** — Column names follow one convention

Principle 1: Standard Import Alias

> *"Always import pandas as pd — it's the universal convention."*

The Problem

Inconsistent import styles:

```
# Version A
import pandas
df = pandas.DataFrame({'Name': ["Ali"]})

# Version B
import pandas as p
df = p.DataFrame({'Name': ["Ali"]})

# Version C
from pandas import *
df = DataFrame({'Name': ["Ali"]})
```

Problem: Code is inconsistent across projects and developers.

The Solution

Use the standard convention:

```
import pandas as pd

df = pd.DataFrame({'Name': ["Ali", "Sara"]})
```

Benefits:

- Recognized by every Python data scientist
- Consistent with documentation and tutorials
- Tools and IDEs provide better autocomplete
- Code is immediately recognizable

The Rule

> "Always import pandas as pd"

Good:

```
import pandas as pd
students = pd.DataFrame({...})
data = pd.read_csv("file.csv")
```

Bad:

```
import pandas
students = pandas.DataFrame({...})
```

Principle 2: Dictionary Construction

> *"Use dictionary construction — column names are explicit and self-documenting."*

The Problem

List of lists requires separate column names:

```
# Column names separate from data
data = [
    ["Ali", 85],
    ["Sara", 92],
    ["Ahmad", 78]
]
students = pd.DataFrame(data, columns=['Name', 'Score'])
```

Issues:

- Column names disconnected from data
- Easy to mismatch number of columns
- Less readable

The Solution

Use dictionary construction:

```
# Column names WITH data
students = pd.DataFrame({
    'Name': ["Ali", "Sara", "Ahmad"],
    'Score': [85, 92, 78]
})
```

Benefits:

- Column names are explicit
- Impossible to mismatch column count
- Self-documenting
- Easier to read and maintain

The Rule

> "Prefer dictionary construction for clarity"

Good:

```
products = pd.DataFrame({
    'Product': ["Laptop", "Mouse"],
    'Price': [2500, 45]
})
```

Acceptable (when data comes row-by-row):

```
data = [["Laptop", 2500], ["Mouse", 45]]
products = pd.DataFrame(data, columns=['Product', 'Price'])
```

Principle 3: Verify Shape

> *"Always check shape and head after loading data — verify expectations."*

The Problem

Load data without verification:

```
students = pd.read_csv("students.csv")

# Process immediately without checking
average = students['Score'].mean()
```

What if:

- File is empty?
- Columns are wrong?
- Data is corrupted?

Errors appear later, harder to debug.

The Solution

Verify immediately after loading:

```
students = pd.read_csv("students.csv")

# Verify dimensions
print(f"Loaded {students.shape[0]} rows and {students.shape[1]} columns")

# Preview data
print(students.head())

# Check structure
students.info()

# Now process with confidence
average = students['Score'].mean()
```

The Rule

> "Check shape and head after loading data"

Pattern:

```
# Load
df = pd.read_csv("data.csv")

# Verify
print(df.shape)      # Expected dimensions?
print(df.head())     # Data looks correct?

# Process
result = df['column'].mean()
```

Catches issues early before they propagate.

Principle 4: Consistent Naming

> *"Choose a column naming convention and stick to it."*

The Problem

Mixed naming styles:

```
students = pd.DataFrame({
    'student name': ["Ali", "Sara"],      # lowercase with space
    'Score': [85, 92],                    # Title case
    'final_grade': ["A", "A+"],           # lowercase with underscore
    'IsActive': [True, False]             # PascalCase
})
```

Problems:

- Hard to remember exact capitalization
- Spaces require bracket notation: `df['student name']`
- Inconsistent and unprofessional

The Solution

Choose one convention and apply consistently:

Option 1: Lowercase with underscores (recommended)

```
students = pd.DataFrame({
    'student_name': ["Ali", "Sara"],
    'score': [85, 92],
    'final_grade': ["A", "A+"],
    'is_active': [True, False]
})
```

Option 2: PascalCase

```
students = pd.DataFrame({
    'StudentName': ["Ali", "Sara"],
    'Score': [85, 92],
    'FinalGrade': ["A", "A+"],
    'IsActive': [True, False]
```

Common Naming Conventions

Convention	Example	Pros	Cons
lowercase_underscore	`student_name`	Readable, SQL-like	Longer to type
PascalCase	`StudentName`	Clean, C#-like	Inconsistent with Python
camelCase	`studentName`	Compact	Rare in Python

Recommendation: Use `lowercase_underscore` (snake_case)

- Most common in Python data science
- Matches Python function naming convention
- Easy to read

The Rule

> "One naming convention per DataFrame"

Good:

```
# All snake_case
df = pd.DataFrame({
    'product_name': [...],
    'unit_price': [...],
    'stock_quantity': [...]
})
```

Bad:

```
# Mixed styles
df = pd.DataFrame({
    'ProductName': [...],
    'unit_price': [...],
    'Stock Quantity': [...]
})
```

4 Principles Summary

#	Principle	Key Rule	Prevents
1	**Standard Import**	Always `import pandas as pd`	Inconsistency
2	**Dictionary Construction**	Use dict for clarity	Column mismatch
3	**Verify Shape**	Check after loading	Wrong data assumptions
4	**Consistent Naming**	One convention	Confusion